学习情境 L10：LLM 工程：路由、降级、缓存与成本（6 学时）

载体：DAP-v2 LLM 网关（LiteLLM）——主备模型路由（qwen2.5:14b 主 / qwen2.5-coder:7b 备 / 确定性模板兜底）+三级缓存（L1 进程真 LRU / L2 Redis TTL 分级 / L3 Milvus 语义缓存）+ TBP 超时预算传播 +熔断器（失败率阈值 OPEN→半开试探）+ DPC 降级契约测试。

*核心命题：**任何一次 NL 查询，5 秒内必有响应；同题重复问询，不允许反复烧 GPU。***

一、学习情境基本信息

项目	内容
学习领域	LF3 智能查询服务开发：NL2SQL 与知识引擎（本域收官讲：LLM 工程化）
讲次	第 10 讲（第 10 周）
学时	6 学时 = 校内理实一体 4 学时（六步法 240’）+企业实训站 2 学时（PR 实战）
布鲁姆主层级	**分析 / 评价**（渗透：理解、应用）
双元场地	校内：理实一体教室（演示区 + 工位区）；企业站：10.167.2.175/176 双节点 K8S 集群 · dap-v2 课程分支 `edu/2026-spring`
企业站 PR 任务	每组认领一条降级路径，注入对应故障、采集日志与 metrics 证据，并补一条 DPC 契约测试提 PR
课后费曼任务	学生讲堂④（录音版）：把 DPC 降级铁律——"没被契约测试证明的降级路径等于不存在"——讲给外行听

二、任务描述

【委托单 #R-2410】委托方：平台运维组。大模型接口高峰期偶发超时与宕机，一次 NL 查询等 56 秒最后还报 500，业务方体验极差；同时 GPU 预算要求压降推理成本。请给主模型配好备胎与确定性兜底，保证任何情况下接口 5 秒内必有响应；同题重复问询不得反复调用模型。——DAP-v2 SLO 改造任务单

三、学习目标

1. 【理解】画出"主→备→模板"三级降级矩阵与触发条件（超时 / 失败率 / 预算耗尽），说出每一级的响应语义与用户可见差异（`degraded` 标记）。
2. 【应用】通过同题二次请求实验，用控制变量法区分 L1 / L2 / L3 三层命中（重启进程灭 L1、改写问法绕精确键），并以 metrics 的 `layer` 标签逐层验证。
3. 【分析】拆解熔断器 CLOSED→OPEN→HALF_OPEN 状态机的转移条件，解释"熔断既救下游也救自己"（快速失败省预算、半开试探防误恢复）。

4. 【分析】结合 TBP 超时预算传播，解释总预算 5s 如何在主模型 3.5s / 备模型 1.2s 之间分配，剩余预算不足时为什么直接降级而不再发起下游调用。

5. 【评价】核算缓存命中率对推理成本的节省（L1 命中省 100% API 调用），并对照 DPC 铁律审查降级路径测试覆盖：任何没被契约测试证明的降级路径视为不存在。

四、教学重点与难点

教学重点

1. 三级降级矩阵（主 qwen2.5:14b → 备 qwen2.5-coder:7b → 确定性模板）与三级缓存命中链（L1 进程 LRU → L2 Redis → L3 Milvus 语义）的结构与验证方法。

2. 熔断器状态机与 DPC 契约测试：降级路径必须被测试证明"被触发且行为正确"。

教学难点（附突破策略）

1. **区分"命中层"**：L1/L2/L3 命中的实验现象相近（都是"变快"），学生容易混淆。

- 突破策略：控制变量实验设计——①同进程同题两次 = L1；② `rollout restart` 灭进程缓存后同题 = L2；③换同义问法绕过精确键 = L3；每步都看 `llm_cache_hits_total{layer=...}` 是否 +1，用指标标签而非体感下结论。

2. **半开（HALF_OPEN）试探的语义**：为什么冷却后只放行 1 个请求、试探失败为什么要立刻回 OPEN。

- 突破策略：状态机图 + "棋盘推演表"——学生在任务单上按请求序号逐行推演（请求 i 结果 / 状态 / 去向），把抽象状态机变成可手算的时序表；教师现场注入故障对照推演结果。

五、教学准备

环境（讲师课前 1 天核验）

- `dap` 命名空间 llm-gateway / redis / milvus / ollama（主备模型）Ready；课程故障注入开关可用：`POST /admin/chaos`（仅课程环境开放，target=primary/fallback/all，mode=timeout/error）。
- Prometheus 可查：`llm_cache_hits_total{layer}`、`llm_cache_misses_total`、`circuit_breaker_state`、`route_chain_total`。
- 课程分支 `edu/2026-spring`；analyst Token 同 L09。

资料

- 代码：`dap-v2/services/llm-gateway/config.yaml`（路由/预算/熔断配置）、`router.py`（主备路由 + TBP 预算切分）、`cache.py`（三级缓存，L1 为 OrderedDict 真 LRU）、`breaker.py`（熔断器）。
- 测试：`dap-v2/tests/llm/test_degradation_contract.py`（DPC 契约测试）。
- 文档：SAD 第 6 章"LLM 网关"、RUNBOOK"模型故障处置"。

分组：4 人一组，角色轮换：操作员 / 指标记录员 / 日志观察员 / 推演员（专责状态机表格）。

六、教学过程设计

第 1 步：资讯（30 分钟）

【教师活动】

- 复盘 L09 遗留问题："FAQ 命中是幸运情况，没命中时怎么办？模型挂了呢？"现场注入 `primary=error`，一次请求演示完整降级链。
- 板书两张图：

```
① 三级缓存命中链
请求 ─► L1 进程 LRU(512) ─miss─► L2 Redis(TTL 分级) ─miss─► L3 Milvus 语义(cosine≥0.92) ─miss─► 模型路由
          │命中(<10ms)              │命中(<50ms)              │命中(重排后返回)                  ├─ 主 qwen2.5:14b  预算 3.5s
          ▼                         ▼                         ▼
          ├─ 备 qwen2.5-coder:7b 用剩余预算
          返回                      返回                      返回
          └─ 模板兜底(不调模型,<100ms,degraded=true)

② 熔断器状态机（breaker.py）
[CLOSED] ─窗口 20 内失败率≥50%─► [OPEN] ─冷却 60s─► [HALF_OPEN] ─试探成功─► [CLOSED]
   ▲                              ▲                    │                    │
   └──────────────────────────────┴────试探失败────────┘（回 OPEN 重新冷却）  │
   （OPEN 态不再打主模型，直接走备/模板）◄──────────────────────────────────┘
```

- 发放引导文：委托单 #R-2410 + `config.yaml` 全文 + 实验记录表（命中层/指标标签/耗时三列）。

【学生活动】

- 通读 `config.yaml`：找出主/备/模板三级、总预算与各级预算、熔断三参数（failure_rate=0.5 / window=20 / 冷却 60s）。
- 回答引导文 3 问：①为什么备模型要"用剩余预算"而不是自己的独立超时？②模板兜底为什么必须打 `degraded=true` 标记？③ L2 的 TTL 为什么要分级（对话 1h / 代码 24h / 知识 7d）？

【教学方法】演示法 + 引导文法。

【设计意图】从上一讲的"幸运命中"自然过渡到"不幸时的工程保障"；两张图分别对应本讲两大主线（缓存省钱、降级保命）。

第 2 步：计划（30 分钟）

【教师活动】给出实验矩阵模板与安全红线（chaos 开关用完必须关、restart 每组限 1 次并提前向教师报备）。

【学生活动】小组分工排期，产出含 4 个实验的计划表：

实验	操作	预期指标变化	预期耗时
降级链	chaos primary=timeout	route_chain 主→备	备路径 <5s
三级缓存	同题×2 / restart 后同题 / 同义改写	hits{L1}/hits{L2}/hits{L3} 各 +1	10ms/50ms/300ms 级
熔断状态机	连续注入 10 次失败	breaker OPEN→HALF_OPEN	状态转移日志
DPC + 成本	跑契约测试 + 算账	pytest 全绿	—

【教学方法】小组计划法 + 实验设计模板。

【设计意图】控制变量是本讲成败关键，计划阶段就把"每一步改哪个变量、看哪个指标"写死，防止实施时变量混杂。

第 3 步：决策（20 分钟）

【教师活动】逐组答辩，拷问两点：①你的实验怎么排除"L2 命中被误认为 L1 命中"？（看指标标签，不看体感。）②熔断推演表里，第 11 个请求（OPEN 态）会打到主模型吗？为什么这反而是优点？

【学生活动】陈述→质询→修订计划表并签字。

【教学方法】专业对话。

【设计意图】把"控制变量""快速失败"两个易错点在动手前锁定；决策记录留档供检查环节回看。

第 4 步：实施（90 分钟）

【教师活动】巡回指导；统一控制 chaos 开关节奏（避免多组互相干扰，按组号分时注入）；收集各组指标截图。

【学生活动】按计划执行，指标记录员把每个实验的 `layer` 标签与耗时写入实验矩阵表。

【教学方法】任务驱动 + 控制变量实验法。

【设计意图】本讲所有结论都必须来自指标（metrics label），训练"用监控数据说话"的运维素养。

4.1 降级矩阵：注入故障观察主→备→模板（20′）

```
make gateway-forward && GW=http://localhost:8000

# ① 注入主模型超时
curl -s -X POST "$GW/admin/chaos" -d '{"target":"primary","mode":"timeout"}'
time curl -s "$GW/api/query" -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H 'Content-Type: application/json' -d '{"question":"按省份统计上月销量"}'
# 预期：总耗时 <5s；日志显示主 3.5s 预算烧完 → 切备 → 备成功
kubectl -n dap logs deploy/llm-gateway --tail=30 | grep route_chain
# route_chain: primary(qwen2.5:14b)=timeout(3500ms) -> fallback(qwen2.5-coder:7b)=ok(9xx ms)

# ② 双模型全灭 → 确定性模板兜底
curl -s -X POST "$GW/admin/chaos" -d '{"target":"all","mode":"error"}'
time curl -s "$GW/api/query" ... -d '{"question":"按省份统计上月销量"}'
# 预期：<100ms 返回；"degraded": true；"data" 结构与正常响应完全同构（sql/rows/columns）
curl -s -X POST "$GW/admin/chaos" -d '{"target":"none"}'   # 用完必关！
```

读 `router.py` 关键段（TBP）：`剩余预算 = deadline - now`；`剩余预算 < 800ms` 时不再发起下一跳，直接落模板——"下游已无预算就别浪费工作"。

4.2 三级缓存命中实验：控制变量三连（30′）

```
Q='按品类统计上个月的 GMV'
# ① 同进程同题两次：第 2 次 L1 命中
curl -s "$GW/api/query" ... -d "{\"question\":\"$Q\"}" > /dev/null   # miss → 模型
time curl -s "$GW/api/query" ... -d "{\"question\":\"$Q\"}"          # <10ms，L1 命中
curl -s "http://localhost:9090/api/v1/query" --data-urlencode \
  'query=llm_cache_hits_total{layer="L1"}'                           # +1

# ② 重启网关灭 L1，同题再问：L2 命中（向教师报备后执行）
kubectl -n dap rollout restart deploy/llm-gateway && sleep 30
```

```
time curl -s "$GW/api/query" ... -d "{\"question\":\"$Q\"}"          # <50ms, L2 命中
#   L2 键样例（课程只读账号）：
redis-cli -h <redis-host> --scan --pattern "llm:resp:*" | head -3

# ③ 同义改写绕过精确键：L3 语义命中
time curl -s "$GW/api/query" ... -d '{"question":"上个月每个类目的成交额汇总"}'
kubectl -n dap logs deploy/llm-gateway --tail=50 | grep -E "cache.*L3.*score=0\.9[2-9]"
```

L1 真 LRU 代码导读（`cache.py`， OrderedDict 实现，命中即 `move_to_end`、淘汰 `popitem(last=False)`）：

```
class L1LRU:
    def __init__(self, cap: int = 512):
        self._d, self.cap = collections.OrderedDict(), cap
    def get(self, k):
        if k not in self._d:
            return None
        self._d.move_to_end(k)            # 真 LRU：命中即摘到队尾
        return self._d[k]
    def put(self, k, v):
        if k in self._d:
            self._d.move_to_end(k)
        self._d[k] = v
        if len(self._d) > self.cap:
            self._d.popitem(last=False)   # 淘汰最久未用（队头）
```

任务单思考题：若把 `move_to_end` 删掉会变成什么策略？（FIFO——最常被误当成 LRU 的假 LRU。）

4.3 熔断器状态机：注入失败推演转移（20′）

```
# 窗口 20 内连续 10 次失败 → 失败率 50% 触发 OPEN
for i in $(seq 1 10); do
  curl -s "$GW/api/query" -H "Authorization: Bearer $ANALYST_TOKEN" \
    -H 'Content-Type: application/json' -d "{\"question\":\"熔断推演第${i}次\"}" > /dev/null
done
kubectl -n dap logs deploy/llm-gateway --tail=100 | grep breaker
# breaker: CLOSED -> OPEN (fail_rate=0.55, window=20)
curl -s "http://localhost:9090/api/v1/query" --data-urlencode 'query=circuit_breaker_state'
```

推演员填"棋盘推演表"（请求序号 / 该请求结果 / 触发前状态 / 转移后状态 / 是否打到主模型）：

- 第 1–10 个失败请求：CLOSED，仍打主模型；
- 第 11 个起：OPEN，**不再打主模型**，直接走备（省掉 3.5s 等待）；
- 60s 冷却后：HALF_OPEN，放行 1 个试探请求——成功→CLOSED，失败→回 OPEN 重新冷却。

现场对照：关闭 chaos 后等 60s 再发 1 个请求，观察 `HALF_OPEN -> CLOSED` 日志。

4.4 DPC 契约测试与成本核算（20′）

```
cd dap-v2 && pytest tests/llm/test_degradation_contract.py -v
# 用例（节选）：
#   test_primary_down_fallback_ok      主挂→备接住，200 且非 degraded
#   test_both_down_template_ok         双挂→模板，200 且 degraded=true
#   test_template_output_schema_stable 模板响应结构与正常响应同构（前端无感）
#   test_faq_hit_skips_llm_entirely    FAQ 命中完全不进模型路由
```

契约测试关键断言（为什么"必须显式降级标记 + 同构 schema"）：

```
def test_both_down_template_ok(chaos_all):
    r = client.query("上月 GMV")
    assert r.status_code == 200
    assert r.json()["degraded"] is True                       # 不得冒充正常结果
    assert set(r.json()["data"].keys()) == {"sql", "rows", "columns"}  # 前端不崩
```

成本核算（任务单表格）：班级 30 人 × 20 问/天 × 30 天 = 18000 问；命中率 60% 时省 10800 次推理；按单次 LLM 路径 56s 折算 GPU 时 = 10800×56/3600≈168 GPU·小时/月。结论一句话：**L1 命中省 100% API 调用——缓存是最高级的降级。**

第 5 步：检查（40 分钟）

【教师活动】发布检查清单，重点核验"指标证据链"：每组必须能回答"你怎么知道是 L2 命中而不是 L1？"

【学生活动】自查 + 互查：

1. 降级链日志两条（主→备 / 双挂→模板）+ 对应耗时；
2. 三级缓存各一条 `hits_total{layer}` +1 证据 + 耗时对比；
3. 熔断推演表 15 行以上且状态转移与日志一致；
4. HALF_OPEN 试探的日志原文；
5. pytest 输出全绿截图；
6. 成本核算表数字自洽；
7. chaos 开关确认已全部关闭（`{"target":"none"}` 返回 200）——**环境复原是硬性检查项**。

【教学方法】核对表法 + 同伴互评。

【设计意图】"看指标不看体感"是本讲的检查主线；环境复原项培养学生"故障注入必须可回滚"的工程纪律。

第 6 步：评价＋学生讲堂（30 分钟）

【教师活动】主持讲堂（抽 3 组各 4′）+ 评分 + 布置任务。

【学生活动】讲堂主题抽签二选一：①"我的降级矩阵推演：从第 1 个到第 11 个请求"；②"缓存是最高级的降级——算一笔 GPU 账"。互评量表打分。

【教学方法】费曼法 + 同伴互评。

【设计意图】状态机推演与成本账是本讲两大可讲述成果；为课后 DPC 铁律宣讲做铺垫。

七、企业站实践任务卡（2 学时）

项目	内容
任务目标	认领一条降级路径，完成"故障注入→现象采集→契约测试补齐"的完整闭环 PR
场地/协作	企业实训站；2 人结对；企业导师按"测试是否真在 CI 跑"把关
认领清单（每组一条）	① 主超时→备 ② 备超时→模板 ③ L1 miss→L2 ④ L2 miss→L3 ⑤ 熔断 OPEN 直走备 ⑥ FAQ 命中跳过模型
步骤	① 通读该路径代码与现有测试，找出未覆盖分支 →

	② 设计注入方案（chaos 开关/重启/改写问法）→ ③ 执行并采集日志 + metrics 证据 → ④ 在 `tests/llm/test_degradation_contract.py` 补一条契约测试（断言触发条件 + 响应语义）→ ⑤ 本地与 CI 双跑绿 → ⑥ 提 PR
交付物	PR 含：新增测试代码 + 故障注入复现步骤（写入 PR 描述） + 证据日志/metrics 截图
评审标准	① 测试在 CI 实际执行（附 CI 链接，防"配而未接"）② 断言含降级语义（degraded 标记 / schema 同构 / 不打主模型之一）③ 注入步骤可复现且已复原 ④ 与现有 57 条 FAQ 回归无冲突
双导师签栏	企业导师（签名/日期）：_____ 校内导师（签名/日期）：_____

八、评价设计（任务单评分表，满分 100）

维度	权重	评价标准
资讯与计划	15	两张图（缓存链/状态机）默画正确；实验矩阵含指标标签列
实施完成度	35	降级链证据（10）；三级缓存逐层指标证据（15）；熔断推演表与日志一致（10）
分析深度	20	TBP 剩余预算逻辑解释正确（8）；半开试探语义推演无误（12）
检查与证据	15	指标证据链完整；环境复原确认；未验证项如实标注
学生讲堂表达	15	推演/算账讲得清、有数据支撑、4 分钟内完成

等级：≥90 优秀 / 75–89 良好 / 60–74 合格 / <60 重做（限补一次）。

九、课后费曼任务

- **讲给谁**：室友或家人（完全不懂分布式系统）。
- **讲什么**：DPC 降级铁律——"没被契约测试证明的降级路径等于不存在"。用消防通道类比：大楼（系统）修了备用出口（降级路径），但从没演练过（契约测试），火灾那天（主模型挂了）你敢赌它是通的吗?
- **自评三问**：①我能说清"降级路径存在"和"降级路径被证明可用"的区别吗? ②对方能复述"为什么要演练"吗? ③我举的三个降级例子（主→备 / 双挂→模板 / OPEN 直走备）各对应什么故障吗? ——任一问答"否"，重讲并录音自查。

十、教学反思栏（授课后 48 小时内填写）

反思维度	现象记录	原因分析	改进措施
目标达成度（5 条目标逐条）			
难点突破效果（命中层区分/半开语义）			
时间分配偏差（哪一步超时）			
学生典型错误与误解			
故障注入对共享环境的干扰			
下一讲迭代事项（衔接 L11 质量规则）			